Cours 3

Vérification déductive

Logique de Hoare

Annotations JML : conception par contrats pour Java

Logique du premier ordre

Java et la logique dynamique

C/ACSL et Frama-C, avec N. Kosmatov (2 cours)

Preuve de programmes Java avec boucles et tableaux

Génération d'obligations de preuves, de Java/JML à JavaDL

Vérification incrémentale

Preuve interactive

Cours 3

Annotations JML : conception par contrats pour Java

- Découverte du langage JML, pour annoter des programmes Java avec des spécifications formelles
- Adaptation de diapositives
  - "Design by Contract with JML", Y. Cheon and G. Leavens, 24 July 2009
    http://sourceforge.net/p/jmlspecs/code/HEAD/tree/TeachingMaterials/trunk/Miscellaneous/jmldbc_slides.ppt
  - "Formal Specification And Verification", P. H. Schmitt (KIT), 2011
    http://i12www.ira.uka.de/~pschmitt/FSS/Folien10/05JML.pdf
  - "Formale Grundlagen der Informatik 3. The Java Modeling Language, Part I", R. Hähnle (TU Darmstadt), 17 December 2013

Ce document est protégé par des droits d'auteur et ne peut être traité, reproduit ou partagé par des systèmes automatisés sans autorisation explicite de l'auteur. Toute utilisation non autorisée par une intelligence artificielle est strictement interdite.

Specifications as Contracts

Useful analogy to stress the different roles/obligations/responsibilities in a specification:

Specification as a contract (between method implementor and user)

"Design by Contract" methodology (Meyer, 1992, EIFFEL)

Contract between *caller* and *callee* (called method)

Callee guarantees certain outcome provided caller guarantees prerequisites

Contracts

Example 1

```
/*@ requires x >= 0.0;
  @ ensures JMLDouble.approximatelyEqualTo(x, \result * \result, eps); */
public static double sqrt(double x) { ... }
```

	Obligations	Rights
Client	Passes non-negative number	Gets square root approximation
Implementor	Computes and returns square root	Assumes argument is not negative

Running Example: ATM.java

```
public class ATM {

    // fields:
    private BankCard insertedCard = null;
    private int wrongPINCounter = 0;
    private boolean customerAuthenticated = false;

    // methods:
    public void insertCard(BankCard card) { ... }
    public void enterPIN (int pin) { ... }
    public int accountBalance() { ... }
    public int withdraw(int amount) { ... }
    public void ejectCard() { ... }
}
```


Informal Specification

Very informal specification of *`enterPIN(int pin)`*

> *"Enter the PIN that belongs to the currently inserted bank card into the ATM. If a wrong PIN is entered three times in a row, the card is confiscated. After having entered the correct PIN, the customer is regarded as authenticated."*

Becoming More Precise: Specification as Contract

Contract states what is guaranteed under which conditions

precondition	card is inserted, user not yet authenticated, card is valid, PIN is correct
postcondition	user is authenticated
precondition	card is inserted, user not yet authenticated, `wrongPINCounter` < 2, PIN is incorrect
postcondition	`wrongPINCounter` is increased by 1 user is not authenticated
precondition	card is inserted, user not yet authenticated, `wrongPINCounter` >= 2, PIN is incorrect
postcondition	card is confiscated, card is made invalid user is not authenticated

Meaning of Pre-/Post-Condition Pairs

Definition

A **pre-/post-condition** pair for a method `m` is **satisfied by the implementation** of `m` if:

> *When `m` is called in any state that satisfies the precondition then in any terminating state of `m` the postcondition is true.*

Remarks

1. No guarantee when the precondition is not satisfied
2. Termination may or may not be guaranteed
3. Terminating state may be reached by normal or by abrupt termination (e.g., exception)

Java Modeling Language (JML)

JML is a *specification language* tailored to JAVA

General JML Philosophy

Integrate

- specification
- implementation

in *one single language*

JML is not external to JAVA, but an *extension* of JAVA

JML
is
JAVA + *FO Logic* + *pre-/post-conditions, invariants* + more ...

JML Annotations

JML *extends* JAVA by *annotations*

JML annotations include:

- ✔ preconditions
- ✔ postconditions
- ✘ class invariants
- ✘ additional modifiers
- ✘ "specification-only" fields
- ✘ "specification-only" methods
- ✔ loop invariants
- ✔ ...
- ✘ ...

✔: in this course, ✘: not in this course

JML/Java integration

JML annotations are attached to JAVA programs
by
writing them directly into the JAVA source code files

Ensures compatibility with standard JAVA compiler:

JML annotations live in special JAVA comments,
ignored by JAVA compiler, recognized by JML tools

JML as Java Comments

From the file `ATM.java`

⋮

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @*/
public void enterPIN(int pin) { ... }
```

⋮

Everything between `/*` and `*/` is invisible for JAVA compiler

JML as Java Comments

JAVA comment lines starting with @ read and parsed by JML tools

```
/*@ public normal_behavior
  @ requires !customerAuthenticated; @ only to beautify
    requires pin == insertedCard.correctPIN;
  @*/
//@ ensures customerAuthenticated; rest-of-line comment
//␣@ ensures !customerAuthenticated; no JML: @ not first
public void enterPIN(int pin) { ... }
```


Informal Description

- Enclosed by (* and *)
- Treated as a boolean value by JML
- Escape from formality (English in contracts)

```
public class IMath {
 /*@ requires (* x is positive *);
     ensures \result >= 0 &&
     (* \result is an int approximation to square root of x *) */
 public static int isqrt(int x) { .. }
}
```


JML by Example: Preconditions

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @*/
public void enterPIN(int pin) { ... }
```

Specification case has two preconditions (marked by `requires`)

Here:
preconditions happen to be *boolean JAVA expressions*

In general:
preconditions are *boolean JML expressions* (including quantifiers)

JML by Example: Preconditions Cont'd

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @*/
```

Both preconditions must be true in prestate

Equivalent to:

```
/*@ public normal_behavior
  @ requires (    !customerAuthenticated
  @             && pin == insertedCard.correctPIN );
  @ ensures customerAuthenticated;
  @*/
```

JML by Example: Postconditions

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @*/
public void enterPIN(int pin) { ... }
```

Specification case has one *postcondition* (marked by `ensures`)

- Postconditions are *boolean JML expressions*
- If there is more than one `ensures` clause: postcondition is the *conjunction* of all clauses

JML by Example: Access of Prestate

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin != insertedCard.correctPIN;
  @ requires wrongPINCounter < 2;
  @ ensures wrongPINCounter == \old(wrongPINCounter) + 1;
  @*/
public void enterPIN(int pin) { ...
```

Access to value of prestate in postcondition

`\old(E)` means: *E* evaluated in the prestate (of `enterPIN()`)

- `\old(E)` is a JML expression that is *not* a JAVA expression
- *E* can be any (arbitrarily complex) JAVA/JML expression

Specification Cases Complete?

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @*/
```

What does specification case *not* tell about poststate?

Fields of class `ATM`:

```
insertedCard
customerAuthenticated
wrongPINCounter
```

What happens with `insertedCard` and `wrongPINCounter`?

Completing Specification Cases

```
/*@ public normal_behavior
  @ requires !customerAuthenticated;
  @ requires pin == insertedCard.correctPIN;
  @ ensures customerAuthenticated;
  @ ensures insertedCard == \old(insertedCard);
  @ ensures wrongPINCounter == \old(wrongPINCounter); */
```

- Similar postconditions added for the other specification cases
- Assumption that environment is unchanged unless explicitly stated: usually called *frame condition*

Clearly unsatisfactory to add

```
  @ ensures loc == \old(loc);
```

for all locations loc which do not change

Assignable Locations

More efficient to explicitly list all locations that *may* change:

`assignable` $loc_1, \ldots, loc_n$;

Assignable clause: value of no location besides $loc_1, \ldots, loc_n$ can change (but could change *temporarily* during execution of method)

Comma-separated list of:

- `e.f` (where `f` is a field)
- `a[*]`, `a[x..y]` (where `a` is an array expression)
- `\nothing` (no location may be changed)
- `\everything` (unrestricted: anything can change, default)

Specification Case with Assignable

```
/*@ public normal_behavior
    requires insertedCard != null;
    requires !customerAuthenticated;
    requires pin != insertedCard.correctPIN;
    requires wrongPINCounter >= 2;
    ensures insertedCard == null;
    ensures \old(insertedCard).invalid;
    assignable wrongPINCounter,
               insertedCard,
               insertedCard.invalid; */
```


Specification of Divergence

`diverges` e;

with a Boolean JML expression e specifies that the method may **not terminate** only when e is true in the pre-state.

Examples

- `diverges false;`
  The method must always terminate.
- `diverges true;`
  The method may terminate or not.
- `diverges n == 0;`
  The method must terminate when called in a state with $n \neq 0$.

JML Modifiers: `pure`

Specifications more concise with *method calls inside JML annotations*

Example

- `o1.equals(o2)`
- `li.contains(elem)`
- `li1.max() < li2.min()`

Specifications may not themselves change the state!

Definition (Pure method)

A JAVA method is called *pure* iff it has no side effects and it always terminates. Specifically, it may create no new objects.

JML expressions may call pure methods. These are annotated by `pure`

```
public /*@ pure @*/ int max() { ... }
```

JML Modifiers: pure Cont'd

How do we know that a pure method is really pure?

- pure puts obligation on implementor not to cause side effects
- It is possible to *formally verify* that a method is pure
  - Write a contract that expresses purity and verify it
- Assignable clauses can be local to a specification case while pure fixes behavior of a method

pure = diverges false; + assignable \nothing;

Specification of Exceptions

```
/*@ requires r;      // what are the caller's obligations
  @ assignable a;    // which locations may be assigned by m
  @ diverges d;      // when may m non-terminate
  @ ensures post;    // what must hold on normal termination
  @ signals_only E1,...,En; // which exception types may be thrown
  @ signals(E e) s; // what must hold when e of type E is thrown
  @*/
T m(...);
```

Abbreviations:

```
normal\_behavior = signals(Exception) false;

exceptional\_behavior = ensures false;
```


JML Expressions ≠ Java Expressions

Definition (JML Expressions – to be completed)

- Each side-effect free JAVA expression is a JML expression
  - E.g., i++ is *not* a JML expression
  - Any method call must be to pure method
  - \result is method's return value
- If *E* is a side-effect free JAVA expression, then \old(*E*) is a JML expression
- If a and b are boolean JML expressions then
  - !a ("not a")
  - a && b ("a and b")
  - a || b ("a or b")
  - a ==> b ("a implies b")
  - a <==> b ("a is equivalent to b")

  are also boolean JML expressions

But this is not enough!

Beyond boolean Java expressions

How to express the following?

- An array "int[] a" contains only non-negative elements
- The variable m holds a maximal element of array a
- All Account objects in the array accountProxies are stored at the index corresponding to their respective accountNumber field
- All created instances of class BankCard have different cardNumbers

Quantified JML Expressions

Definition (JML Expressions)

- Each side-effect free JAVA expression is a JML expression
- If E is a side-effect free JAVA expression, then `\old(E)` is a JML expression
- If `a` and `b` are `boolean` JML expressions, *x is a variable of type t*:
  - `!a` ("not a")
  - `a && b` ("a and b")
  - `a || b` ("a or b")
  - `a ==> b` ("a implies b")
  - `a <==> b` ("a is equivalent to b")
  - `(\forall t x; a)` ("for all x of type t, a is true")
  - `(\exists t x; a)` ("there exists x of type t such that a")
  - `(\forall t x; a; b)` ("for all x of type t fulfilling a, b is true")
  - `(\exists t x; a; b)` ("there exists an x of type t fulfilling a, such that b is true")

  are also `boolean` JML expressions.

Range Predicates

JML quantifiers (optionally) have more general syntax than FOL ones

Definition (Range predicate)

In the JML expressions `(\forall t x; a; b)` and `(\exists t x; a; b)` the `boolean` `a` is called *range predicate*.

Range predicates are syntactic sugar for standard FOL quantifiers:

`(\forall t x; a; b)`
equivalent to
`(\forall t x; a ==> b)`

`(\exists t x; a; b)`
equivalent to
`(\exists t x; a && b)`

Pragmatics of Range Predicates

Range predicates used to restrict *range* of `x` further than to its type `t`

Example

"Array a is sorted between indices 0 and 9":

```
(\forall int i,j; 0<=i && i<j && j<10; a[i] <= a[j])
```


Using Quantified JML Expressions

- All `Account` objects in the array `accountProxies` are stored at the index corresponding to their respective `accountNumber` field

```
(\forall int i; 0 <= i && i < maxAccountNumber;
                accountProxies[i].accountNumber == i )
```

- All created instances of class `BankCard` have different `cardNumbers`

```
(\forall BankCard b1, b2;
         \created(b1) && \created(b2);
         b1 != b2 ==> b1.cardNumber != b2.cardNumber)
```

Remarks

- Restrict range to created objects with JML keyword `\created`
- JML/KeY quantifiers range even over non-created objects

Generalized Quantifiers

- `\max`
- `\min`
- `\product`
- `\sum`

returning the *maximum* resp. *minimum*, *product*, or *sum* of the values of the expressions given, where the variables satisfy the given range.

Examples (all formulae are `true`):

```
(\sum int i; 0 <= i && i < 5; i) == 0 + 1 + 2 + 3 + 4
(\product int i; 0 < i && i < 5; i) == 1 * 2 * 3 * 4
(\max int i; 0 <= i && i < 5; i) == 4
(\min int i; 0 <= i && i < 5; i-1) == -1
```


JML assertions

Clause

- `assert` F;

where F is any JML formula

Example:

```
/*@ ensures \result == n+1; */
public static int incr(int n) {
  n = n+1;
  /*@ assert n == \old(n)+1; */
  return n;
}
```

It's much better to test with assertions than to try to look at output!
You can use Java's assert too, but there are advantages to using JML's assert, as is done here.

Loop annotations loop_invariant, assignable and decreases

First explanations, more details later...

Example – `Loop.java`

```
/*@ ensures
      (\forall int x; 0 <= x && x < a.length; a[x] == 1); */
public void m(int[] a) {
  int i = 0;
  /*@ loop_invariant ???;
      assignable ???;
      decreases ???; */
  while(i < a.length) {
    a[i] = 1;
    i++;
  }
}
```


1. Accurate Specification of Modified Locations

- `assignable` *clause* for loops tell what the loop can possibly modify

```
/*@ diverges true; */
public void m(int[] a) {
  int i = 0;
  /*@ loop_invariant true;
      assignable i, a[*]; */
  while(i < a.length) {
    a[i] = 1;
    i++;
  }
}
```

Demo

2. Justify Termination

Find a decreasing integer term v (called *variant*) such that

- $v \geq 0$ is initially valid
- $v \geq 0$ is preserved by the loop body
- v is strictly decreased by the loop body

Proving termination in JML/Java

- Remove directive `diverges true;` from contract
- Add directive `decreases v;` to loop invariant

2. Justify Termination, Example

```
/*@ ensures true; */
public void m(int[] a) {
  int i = 0;
  /*@ loop_invariant true;
      assignable i, a[*];
      decreases a.length-i; */
  while(i < a.length) {
    a[i] = 1;
    i++;
  }
}
```

Demo

3. Justify Some Postcondition

```
/*@ ensures
      (\forall int x; 0 <= x && x < a.length; a[x] == 1); */
public void m(int[] a) {
  int i = 0;
  /*@ loop_invariant ???;
      assignable i, a[*];
      decreases a.length-i; */
  while(i < a.length) {
    a[i] = 1;
    i++;
  }
}
```

Stronger loop invariant :

..... $[R_{24}]$

```
loop_invariant i <= a.length;
loop_invariant (\forall int x; 0 <= x && x < i; a[x] == 1);
```


Summary

```
/*@ requires ..;
  @ assignable ..;
  @ ensures .. \result ..;
  @ diverges ..; */
t1 m(t2 arg) {
 t3 tmp;
 /*@ loop_invariant ..;
   @ assignable ..;
   @ decreases ..; */
 for (.. ; ..; ..) ..
 /*@ assert ..; */
 return(tmp);
}
```

Defaults for omitted clauses

- `requires true;`
- `assignable \everything;`
- `ensures true;`
- `diverges false;`
- `loop_invariant true;`